

BEYAN

Bu raporda sunulan «MongoDB GeoSpatial API» başlıklı çalışmanın tamamen tarafımdan hazırlanmış, bilimsel etik kurallarına uygun olarak yürütülmüş ve tüm alıntılar ile kaynaklar kaynakça bölümünde açıkça belirtilmiş olduğunu beyan ederim. Geliştirme sürecinde yapay zekâ destekli araçlardan yardım alınmış olmakla birlikte; mimari kararlar, kullanılan teknoloji seçimleri, ürün yönetimi belgesi (PRD), kullanıcı araştırması ve iş modeli değerlendirmeleri tamamen kendi araştırmam ve takdirim sonucu oluşturulmuştur. AI araçlarından üretilen kod parçaları gözden geçirilmiş, gerekli iyileştirmeler yapılmış ve proje bütününe entegre edilmiştir.

ÖZET

MONGODB GEOSPATIAL API

Günümüzde kullanıcılar bulundukları lokasyona en yakın kafe, restoran ve çalışma alanlarını hızlı ve kamaşadan uzak bir şekilde bulma ihtiyacı hissetmektedir. Kapsamlı harita uygulamaları (örneğin Google Maps) genellikle aşırı bilgi yüklemesi yaratmakta ve lokal odaklı "hemen şuradaki mekan" (hyperlocal) aramalarında hantal kalabilmektedir. Bu proje, "YakınKafe" adıyla, kullanıcıların 5 km çapındaki mekanları anında harita üzerinde görüntülemesini, yeni mekanlar eklemesini ve yönetici onay sürecinden geçmesini sağlayan bir platform sunarak bu problemi çözmektedir.

Proje, modern ve performans odaklı bir teknoloji yığını ile geliştirilmiştir. Backend tarafında Node.js 20 ve Fastify kullanılarak yüksek hızlı bir API mimarisi kurulmuş, veritabanı olarak MongoDB tercih edilmiştir. Harita ve mesafe hesaplamaları için PostGIS gibi ağır ilişkisel çözümler yerine, MongoDB'nin yerleşik 2dsphere indeksi ve \$geoNear / \$geoWithin operatörleri kullanılarak yüksek performanslı mekansal (geospatial) sorgular entegre edilmiştir. Frontend tarafında ise React 18, Vite ve Tailwind CSS kullanılarak "Glassmorphism" tasarımlı, premium ve mobil uyumlu bir arayüz geliştirilmiştir. Harita altyapısı Leaflet ve CartoDB Voyager teması ile desteklenmiştir.

Sonuç olarak, kullanıcıların konumlarını anlık olarak alıp çevrelerindeki onaylı mekanları listeleyen, gizli admin paneli ve Rol Tabanlı Yetkilendirme (RBAC) ile güvenliği sağlanan tam teşekküllü bir SPA (Single Page Application) ortaya konmuştur. Koordinat uyumsuzlukları ve seed verilerin göl üzerine düşmesi gibi coğrafi problemler algoritmik düzenlemelerle çözülmüş olup, projenin turizm ve yerel işletme keşfi dikeylerinde ileriye dönük ticari potansiyeli yüksektir.

Anahtar Kelimeler: Konum Tabanlı Servisler, Node.js 20 + Fastify, MongoDB 7 + Mongoose, 2dsphere index, React 18 + Vite, RBAC, PRD

ABSTRACT

MONGODB GEOSPATIAL API

Today, users need to find nearby cafes, restaurants, and study areas quickly and without clutter. Comprehensive mapping applications often create information overload and can be cumbersome for hyperlocal "right around the corner" searches. This project solves this problem by providing a platform named "YakınKafe", which allows users to instantly view places within a 5 km radius on a map, add new places, and pass them through an administrative approval process.

The project was developed with a modern and performance-oriented technology stack. On the backend, a high-speed API architecture was established using Node.js 20 and Fastify, with MongoDB chosen as the database. Instead of heavy relational solutions like PostGIS for map and distance calculations, highly performant geospatial queries were integrated using MongoDB's built-in 2dsphere index and \$geoNear / \$geoWithin operators. On the frontend, a premium, mobile-responsive interface featuring a "Glassmorphism" design was developed using React 18, Vite, and Tailwind CSS. The mapping infrastructure is supported by Leaflet and the CartoDB Voyager theme.

As a result, a fully-fledged SPA (Single Page Application) was created that instantly captures users' locations, lists approved places around them, and ensures security with a hidden admin panel and Role-Based Access Control (RBAC). Geographic problems such as coordinate mismatches and seed data falling over the lake were solved with algorithmic adjustments, highlighting the project's high future commercial potential in tourism and local business discovery verticals.

Keywords: Location-Based Services, Node.js 20 + Fastify, MongoDB 7 + Mongoose, 2dsphere index, React 18 + Vite, RBAC, PRD

İÇİNDEKİLER

İçindekiler güncellemek için F9

ŞEKİLLER LİSTESİ

Şekiller listesi güncellemek için F9

TABLolar LİSTESİ

Tablolar listesi güncellemek için F9

KISALTMALAR VE SİMGELER

Kısaltma Açıklama

API	Application Programming Interface — Uygulama Programlama Arayüzü
CRUD	Create, Read, Update, Delete
DB	Database — Veritabanı
ERD	Entity-Relationship Diagram — Varlık-İlişki Diyagramı
JWT	JSON Web Token
NFR	Non-Functional Requirements — İşlevsel Olmayan Gereksinimler
PRD	Product Requirements Document
SPA	Single Page Application
RBAC	Role-Based Access Control - Rol Tabanlı Erişim Kontrolü
Lng/Lat	Longitude (Boylam) / Latitude (Enlem)
SWOT	Strengths, Weaknesses, Opportunities, Threats

1. GİRİŞ

1.1. Problem Tanımı

Cözölen problem (proje tanımı): 'Bana en yakın kafeler', 'yakındaki etkinlikler' gibi sorgular için PostGIS overkill; MongoDB 2dsphere basit ve performanslı.

Günlük hayatta öğrenciler, uzaktan çalışanlar veya turistler bulundukları konuma en yakın, nitelikli bir kafe veya restoran arayışına sıkça girmektedir. Google Maps veya Foursquare gibi uygulamalar devasa veritabanlarına sahip olsalar da arayüzleri karmaşıktır ve "sadece 5 km yakınımdaki iyi kafeleri göster" gibi niş bir amacı gerçekleştirirken çok fazla reklam veya alakasız yer (oto sanayi, alakasız işletmeler vb.) gösterebilmektedir. Sosyal platformlardaki kullanıcı yorumları, harita uygulamalarının giderek "sanı sayfalara" dönüştüğünden şikayet etmektedir.

Teknik açıdan ise, basit 'yakınımda' sorguları için tam teşekküllü PostGIS gibi ağır coğrafi sistemler kullanmak bir "overkill" (gereğinden fazla karmaşık) yaklaşımıdır. Bu proje ile amaçlanan; yalnızca MongoDB 2dsphere indekslerini kullanarak yüksek performanslı, hızlı ve karmaşadan uzak bir lokal keşif platformu oluşturmaktır.

1.2. Projenin Amacı ve Kapsamı

Bu projenin temel amacı, Bitlis (merkez ve Tatvan öncelikli olmak üzere) bölgesi merkezli, kullanıcıların anlık konumlarına göre çevrelerindeki işletmeleri (kafe, restoran, çalışma alanı) harita üzerinde listeleyen, premium bir web uygulaması (YakınKafe) geliştirmektir.

Kapsam dahilindekiler (V1): Harita üzerinde mekan gösterme, 5 km yarıçap sorgusu (\$geoNear), polygon sorgusu (\$geoWithin), kullanıcıların mekan önermesi, JWT tabanlı kimlik doğrulama ve RBAC mimarili gizli bir Admin Onay paneli.

Başarı kriterleri: Sorguların (API response) 500ms altında gerçekleşmesi ve UI tarafında kullanıcının tek bir butona ("Konumumu Bul") tıklayarak haritayı filtreleyebilmesidir. Canlı navigasyon (turn-by-turn) kapsamı dışıdır.

1.3. Raporun Yapısı

Bölüm 2'de ürün yönetimi belgesi (PRD) kapsamında kullanıcı araştırması ve gereksinimler sunulmuştur. Bölüm 3'te pazar ve rakip analizi yer almaktadır. Bölüm 4 ve 5, sistemin teknoloji yığını ve mimari kararlarını detaylandırırken, Bölüm 6 veritabanı şemasını ve API yapısını açıklar. Kalan bölümlerde ise kullanıcı arayüzü, güvenlik, test, geliştirme aşamasında yaşanan zorluklar ve elde edilen sonuçlar yer almaktadır.

2. GEREKSİNİM ANALİZİ — PRD

2.1. Yönetici Özeti (Executive Summary)

YakınKafe, lokasyon bazlı ve hiper-yerel (hyperlocal) mekan keşfi sağlayan bir platformdur. Kullanıcıları karmaşık harita uygulamalarından kurtararak doğrudan ilgilendikleri kategorideki (kahve, tatlı, restoran) mekanları harita üzerinde hızlıca sunmayı hedefler.

Özellikle yerel halkın ve üniversite öğrencilerinin nitelikli mekan bulma zorluğu, uygulamanın çıkış noktasıdır. Verilerin kalitesini korumak için kullanıcıların eklediği mekanlar, doğrudan yayına alınmak yerine gizli bir "Admin" portalından onay sürecinden geçmektedir.

Projenin başarısı, uygulamanın günlük kullanım kolaylığı (hızlı konum bulma) ve adminlerin spam içerikleri temizleme yeteneği ile ölçülecektir.

2.2. Hedef Kitle ve Persona

Birincil segment: Yemek/etkinlik/hizmet uygulaması kullanıcıları

Tablo 1 : Persona 1 Kartı (Öğrenci)

Alan	Değer
Ad	Dijital Zehra
Yaş / Şehir	21 / Bitlis (Tatvan)
Rol / Meslek	Üniversite Öğrencisi
Teknoloji kullanımı	Mobil ağırlıklı, iOS kullanıcısı
Günlük rutin	Dersten çıkıp laptop'u ile çalışabileceği sessiz bir kafe arar.
Ana hedef	Hızlıca prize sahip, interneti iyi, yakın bir kahveci bulmak.
Pain Points (3)	1. Haritada alakasız mekanların çıkması; 2. Sadece zincir mekanların görünmesi; 3. Yavaş yüklenen haritalar.
Ürünümüzü ne zaman açar?	Kampüste çıktığı an "yakınımda neresi var?" diyerek.
Motto	"Zamanım kıtlı, en hızlı şekilde kahveni alıp çalışmalıyım."

Tablo 2 : Persona 2 Kartı (Gezgin)

Alan	Değer
Ad	Gezgin Burak
Yaş / Şehir	34 / İstanbul (Bitlis'e seyahatte)
Rol / Meslek	Turist / Fotoğrafçı
Teknoloji kullanımı	Web ve Mobil, Android kullanıcısı
Günlük rutin	Yeni yerler keşfetmek, lokal tatları bulmak.
Ana hedef	Yöresel lezzet sunan, yüksek puanlı restoranlar bulmak.
Pain Points (3)	1. Turistik tuzaklara düşmek; 2. Bölgesel özelliklerin haritalarda görünmemesi; 3. Sahte yorumlar.
Ürünümüzü ne zaman açar?	Yeni bir şehre/ilçeye vardığında yemek saatinde.
Motto	"Gerçek yerel lezzeti, gerçek insanların önerdiği yerlerde bulurum."

2.3. Jobs To Be Done (JTBD)

- When I'm bilmediğim bir mahalledeyken, I want to bana en fazla 5 km uzakdaki kaliteli kafeleri görmek istiyorum, so I can kaybolmadan hızlıca bir kahve molası verebileyim.
- When I'm kendi favori mekanımı haritada bulamadığımda, I want to bu mekanı sisteme önerebilmek istiyorum, so I can başkalarının da bu mekanı keşfetmesine katkı sağlayabileyim.
- When I'm sistem yöneticisiyken, I want to kullanıcıların eklediği mekanları bir panel üzerinden inceleyebilmek istiyorum, so I can sahte/yanlış konumların (öneğin Van Gölü'nün ortası) haritayı kirletmesini engelleyebileyim.

2.4. Ana Özellikler ve Kullanıcı Hikâyeleri (User Stories)

Tablo 3 : MVP Kapsamındaki Temel Özellikler

Özellik	Açıklama / Öncelik
Nokta (POI) CRUD — kafe, restoran, etkinlik	Must-have / V1
Harita üzerinde göster (Leaflet / CartoDB)	Must-have / V1
"Bana 5 km içinde olanlar" sorgusu (\$geoNear)	Must-have / V1
Kategori + filtre (fiyat, puan)	Must-have / V1
Admin Onay / Ret Mekanizması (RBAC)	Must-have / V1
Gizli Admin Login Portalı (/admin/login)	Must-have / V1

- As a öğrenci, I want to sadece tek bir butona tıklayarak konumuma en yakın mekanları haritada görmek, so that hızlıca kahveni içebileyim.
- As a sistem yöneticisi, I want to /admin/login sayfasına girerek sadece onay bekleyen mekanları görmek, so that sahte konumları reddedebileyim.

2.5. İşlevsel Olmayan Gereksinimler (NFR)

Tablo 4 : NFR Matrisi

Kategori	Gereksinim	Nasıl ölçülecek?
Performans	API response < 500 ms (\$geoNear)	MongoDB Profiler

performans	API response < 500 ms (geçerli)	mongoDB driver
Güvenlik	Admin rotalarına sadece yetkili (RBAC) erişim	Postman/Manuel Test
UI/UX	Mobil görünümde haritanın ve Navbar'ın %100 uyumlu olması (Glassmorphism)	Cihaz testleri
Veri Doğruluğu	Mekanların sadece "approved" olanlarının public API'de çıkması	Integration testleri

2.6. Kapsam Dışı (Out-of-Scope)

V1 kapsamında; canlı rota tarifi (tum-by-tum navigation), mekan sahipleri için işyeri sahiplenme (claim business) ekranları ve gerçek zamanlı chat uygulaması kapsam dışı bırakılmıştır.

3. PİYASA VE REKABET ANALİZİ

3.1. Pazar Büyüklüğü (TAM / SAM / SOM)

- TAM (Total Addressable Market): Türkiye'deki tüm akıllı telefon kullanıcılarının lokasyon bazlı arama pazar büyüklüğü.
- SAM (Serviceable Available Market): Aktif olarak yeme-içme ve mekan arayışında olan, Z ve Y kuşağı mobil internet kullanıcıları.
- SOM (Serviceable Obtainable Market): Bitlis, Tatvan ve çevre ilçelerindeki üniversite öğrencileri ve lokal kafe/restoran ziyaretçileri.

3.2. Rakip Karşılaştırma Matrisi

Tablo 5 : Rakiplerle Özellik Karşılaştırması

Özellik	YakınKafe (Bizim)	Google Maps	Foursquare	Zomato
Sade & Sadece Mekan Odaklı UI	✓	✗ (Karmaşık)	✓	✗ (Sadece yemek)
Topluluk Odaklı Ekleme & Admin Onayı	✓	✓ (Hantal)	✓	✓
Hızlı "Etrafımdakiler" Filtresi	✓	✗ (Arama Gerekir)	✓	✗
Glassmorphism Premium Tasarım	✓	✗	✗	✗

3.3. Detaylı Rakip İncelemesi

- Google Maps: Çok kapsamlı, devasa veri. Zayıf yönü: Sadece kahve arayan biri için gereksiz detaylar sunması.
- Foursquare: Lokal keşif aracı. Güçlü yönü: Yorum topluluğu. Zayıf yönü: Türkiye'de kullanım oranının düşmesi.
- Zomato: Yemek ağırlıklı. Zayıf yönü: Sadece restoran menülerine odaklanıp harita deneyimini ikinci plana atması.

3.4. SWOT Analizi

Tablo 6 : SWOT Matrisi

Güçlü Yönler (S)

- Modern teknoloji yığını (Fastify, MongoDB 2dsphere)
• Temiz arayüz (Glassmorphism)
• Gizli admin paneli ve RBAC
- Fırsatlar (O)
- Yerel turizm ve üniversite ağılarına genişleme imkanı
• Mekanların uygulamaya sponsor olması

Zayıf Yönler (W)

- Başlangıçta mekan veritabanının kısıtlı olması
• Native mobil uygulamanın (şimdilik) olmaması
- Tehditler (T)
- Büyük rakiplerin benzer "hiper-yerel" özellikler duyurması
• Yanlış veri girme atakları (RBAC ile minimize edildi)

3.5. Positioning Statement (Konumlandırma)

FOR yeni bir yer keşfetmek isteyen öğrenciler ve gezginler WHO karmaşık haritalar arasında kaybolmaktan sıkılanlar, OUR PRODUCT IS A lokasyon bazlı keşif web uygulaması THAT anında 5 km çapındaki en iyi kafeleri gösterir UNLIKE Google Maps OUR PRODUCT sadece premium, ilgili ve onaylanmış kafe/restoran verilerine odaklanarak temiz bir UI sunar.

4. TEKNOLOJİ YİĞİNİ (TECH STACK)

4.1. Katmanlar — Özet Tablosu

Tablo 7 : Projede Kullanılan Teknoloji Katmanları

Katman Teknolojiler
:-- :--
Backend Node.js 20 + Fastify
Database MongoDB 7 + Mongoose
GeoSpatial MongoDB 2dsphere index & \$geoNear / \$geoWithin
Frontend React 18 + Vite + Tailwind CSS + Framer Motion
Harita Leaflet JS (CartoDB Voyager)
Auth & Security JWT, fastify-jwt, bcrypt

4.2. Backend

4.2.1. Node.js 20 + Fastify

Node.js üzerinde koşan son derece hızlı bir web framework.

Neden Seçildi?: Express'je göre saniyede çok daha fazla request işleyebilen, dahili şema doğrulama (schema validation) yeteneklerine sahip olan Fastify tercih edilmiştir. Harita üzerinden gelecek anlık lokasyon sorgularına çok hızlı yanıt vermek kritik olduğu için asenkron mimarisi kullanılmıştır.

4.3. Database

4.3.1. MongoDB 7 + Mongoose

NoSQL doküman tabanlı modern veritabanı.

Neden Seçildi?: JSON tabanlı veri saklaması JavaScript/Node.js ile direkt uyumluluk sağlar ve şemasız esnekliği hızlı prototiplemeyi kolaylaştırır. Mongoose ile modeller yapılıp güvenlik sağlanmıştır.

4.3.2. 2dsphere index

MongoDB'nin dünya yüzeyindeki lokasyon tabanlı sorgular için yerleşik algoritması.

Neden Seçildi?: Koordinat bazlı sorgular için geleneksel SQL veritabanlarında PostGIS kurulumu ve yönetimi gereklidir. Ancak bizim projemizde ana işlev sadece 5 km çapında ve belirli poligonlar içindeki belgeleri getirmek. MongoDB'nin sunduğu yerleşik GeoJSON desteği ve 2dsphere indekslemesi, bu işlemleri inanılmaz hızlı ve kolay bir şekilde çözmemizi sağlamıştır.

4.4. Frontend

4.4.1. React 18 + Vite

SPA (Single Page Application) oluşturmak için modern kütüphane ve derleyici araç kiti.

Neden Seçildi?: Çok hızlı bir component tabanlı mimari kurabilmek. Harita render işlemleri (Leaflet) ile state'lerin çakışmaması için React'in Lifecycle'ları kullanılmıştır. Vite ile build (derleme) süreleri minimize edilmiştir.

4.5. Harita

4.5.1. Leaflet JS

Hafif ve açık kaynaklı JavaScript harita kütüphanesi.

Neden Seçildi?: Google Maps API'nin maliyetli ve karmaşık olmasından ötürü, açık kaynaklı, hafif ve performansı yüksek Leaflet JS tercih edilmiştir. Harita katmanı olarak daha estetik duran CartoDB Voyager kullanılmıştır.

4.6. Auth

4.6.1. JWT

JSON Web Token üzerinden durumsuz (stateless) yetkilendirme.

Neden Seçildi?: Sunucu tarafında session tutmaya gerek kalmadan, sadece token ile hızlı doğrulama yapar. Admin / User rolleri (RBAC) direkt olarak token payload'unda saklanmıştır.

4.8. Reddedilen Teknoloji Kararları

- PostgreSQL / PostGIS: MVP aşamasındaki basit "5 km mesafe" sorguları için aşırı (overkill) bulundu.
- Express.js: Fastify'nin yüksek performans test sonuçları nedeniyle terk edildi.
- Google Maps API: Ücretlendirme politikaları ve API anahtarı sınırlandırmaları nedeniyle Leaflet tercih edildi.

5. SİSTEM MİMARİSİ

5.1. Yüksek Seviye Mimari (C4 — Level 1: Context)

Kullanıcı (Frontend Tarayıcısı) -> Vite/React SPA -> Fastify REST API -> MongoDB Veritabanı

Sistem dışı bağımlılık olarak harita görsellerini çekmek için Leaflet üzerinden CartoDB/OSM (OpenStreetMap) sunucuları ile iletişim kurar.

5.5. Mimari Karar Kayıtları (Architecture Decision Records — ADR)

Tablo 8 : ADR Özet Listesi

ADR No	Karar	Durum	Tarih
ADR-001	Veritabanı: MongoDB ve 2dsphere kullanımı	Accepted	2026-05-15
ADR-002	Güvenlik: Admin ve Kullanıcı girişlerinin /admin/login rotasıyla ayrılması	Accepted	2026-05-17
ADR-003	Mekan Onay Süreci: Eklenecek mekanların default 'pending' statüsünde olması	Accepted	2026-05-17

6. VERİ MODELİ VE API TASARIMI

6.2. Tablo Tanımları

Tablo 9 : Koleksiyon: Places (Mekanlar)

Kolon	Tip	Özellik	Default	Açıklama
_id	ObjectId	PK	Auto	
name	String	Required	—	Mekan Adı
location	GeoJSON	2dsphere Index	—	{ type: "Point", coordinates: [Lng, Lat] }
status	String	Enum	"pending"	Admin Onay durumu ("pending", "approved", "rejected")
category	String	Enum	—	"kahve", "restoran", "tatli" vs.

Tablo 10 : Koleksiyon: Users (Kullanıcılar)

Kolon	Tip	Özellik	Default	Açıklama
email	String	Unique	—	Login id
password	String	Hashlanmış	—	bcrypt ile
role	String	Enum	"user"	"user" veya "admin" (RBAC için kritik)

6.3. İndeks Stratejisi

Tablo 11 : İndeks Kararları

Tablo	İndeks	Amaç
users	email (unique)	Hızlı login sorgusu
places	location ("2dsphere")	Hızlı coğrafi \$geoNear aramaları
places	status(1)	Public API'de sadece 'approved' filtrelemesi

6.4. REST API Endpoint Listesi

Tablo 12 : Ana API Endpoint'leri

Metot	URL	Açıklama	Auth
GET	/api/places/nearby	En yakın "approved" mekanlar	Public
POST	/api/places	Kullanıcı mekan ekler (status: pending)	JWT (User)
POST	/api/auth/login	Normal kullanıcı girişi	Public
GET	/api/admin/cafes/pending	Onay bekleyen mekanlar	JWT (Admin)
PUT	/api/admin/cafes/:id/approve	Mekanı onayla	JWT (Admin)
PUT	/api/admin/cafes/:id/reject	Mekanı reddet	JWT (Admin)

6.5. Authentication Akışı

Kullanıcılar ve adminler JWT üzerinden doğrulanmaktadır. Kullanıcı admin@yakinkafe.com ile /admin/login sayfasından giriş yaptığında JWT token üretilir. Sunucu, /api/admin/* rotalarında çalışan özel bir adminOnly middleware'ile ile token'ı çözer ve rolün admin olup olmadığını kontrol eder. Bu sayede sisteme sızmaların ve spam koordinat gönderimlerinin önüne geçilir.

7. KULLANICI ARAYÜZÜ TASARIMI

7.3. Design System

- Konsept: Glassmorphism (arkası bulanık cam görünümü), akıcı animasyonlar (Framer Motion).
- Renkler: Amber/Orange (Marka Rengi - #F59E0B), Emerald (Kahve), Rose (Restoran).
- Tipografi: Sans-serif temiz fontlar.

7.5. Gizli Admin Portalı

Normal kullanıcıların kafasını karıştırmamak adına Admin arayüzü ana sayfadan tamamen koparılmıştır.

- /admin/login: Sadece e-posta ve şifre alan koyu (dark) temalı, animasyonlu, premium bir giriş ekranıdır.
- Mekan Ekleme İstekleri: Admin sisteme girdiğinde haritada Navbarda "📍 Mekan Ekleme İstekleri" adında özel bir buton belirir ve onay bekleyen mekanlar panelinden yönetilir.

8. GÜVENLİK, PERFORMANS VE TEST

8.1. Güvenlik (OWASP Top 10)

Tablo 13 : OWASP Top 10 (2021) Uygulama Matrisi

Kod	Risk	Bu projedeki önlemim
A01	Broken Access Control	Her endpoint'te authz middleware + RBAC
A02	Cryptographic Failures	bcrypt cost 10, gizli env secrets
A03	Injection (SQL / XSS)	Mongoose Validation

8.2. Performans Hedefleri

Tablo 14 : Performans Metrikleri ve Hedefler

Metrik	Hedef	Ölçüm aracı
LCP (Largest Contentful Paint) < 2.5 s		Lighthouse
API P95	< 500 ms	Network Sekmesi

9. MALİYET, GELİR MODELİ VE GTM

9.1. Business Model Canvas

Blok	İçerik
Customer Segments	Öğrenciler, gezginler, lokal halk
Value Propositions	En hızlı ve temiz lokasyon tabanlı mekan keşfi.
Revenue Streams	Reklam (Sponsored Places), İşletme Sahiplenme.
Key Resources	MongoDB, Node.js, Frontend React yapısı.

9.2. Gelir Modeli

Freemium modeli ile uygulama tamamen ücretsiz kalır. Ancak ileride mekan sahipleri aylık abonelik ücreti ile "Sponsorlu Onaylı Mekan" veya "Kendi Sayfayı Yönet" paketleri alabilir.

10. UYGULAMA VE GELİŞTİRME

10.1. Kurulum ve Çalıştırma

1) Repoyu klonla

git clone https://github.com/Furkangnrd/yakin_cafe.git

2) Ortam değişkenleri

cp .env.example .env

3) Bağımlılıklar

cd server && npm install
cd client && npm install

4) Çalıştır

npm run dev

10.3. Kullanılan AI Araçları ve Katkı Oranı

Araç	Kullanım Oranı Ne için?
Claude / Gemini (Anthropic/Google)	%60 Mimari Kararlar, Kod üretme, Refactoring
AI çıktısının gözden geçirilmesi	%100 Tüm AI çıktıları kontrol edilmiş, projedeki coordinate Lng/Lat çıkışmaları manuel onanılmıştır.

11. SONUÇ VE DEĞERLENDİRME

11.1. Proje Hakkında Genel Değerlendirme

Proje belirlenen tüm MVP (Minimum Viable Product) hedeflerine ulaşmıştır. Hızlı çalışan, MongoDB GeoSpatial \$geoNear mimarisini başarıyla sergileyen, modern SPA (React) yapısına sahip ve en önemli "Admin / Kullanıcı" yetki hiyerarşisi çok iyi çözülmüş bir platform elde edilmiştir.

11.2. Karşılaşılan Zorluklar ve Çözümleri

Tablo 15 : Geliştirme Sürecindeki En Büyük 3 Zorluk

Zorluk	Çözüm	Öğrenilen
MongoDB vs Leaflet Koordinat Formatı Uyuşmazlığı: MongoDB (Lng, Lat) beklerken Harita (Lat, Lng) beklentisi.	Servis katmanında veriler MongoDB'ye yazılmadan önce ters çevrildi, Frontend okurken düzeltildi.	GeoJSON standartlarının API servisleri arasında her zaman aynı olmadığı.
Sahte/Yanlış Seed Verileri: Algoritmanın mekanları Van Gölü'ne yerleşti.	Rastgelelik kaldırıldı. Gerçek doğrulan koordinatlarla hard-coded seed dosyaları (seedBitlisCafes) hazırlandı.	Test verilerinin coğrafi projelerde rastgele olmaması gerektiği.
UX Ayırımı: Admin özelliklerinin normal kullanıcı arayüzünü karmaşıktırması.	/admin/login adında tamamen izole edilmiş bir portal ve status field bazlı onaylama akışı eklendi.	Kullanıcı deneyimi ile yönetim deneyiminin (RBAC) ayrıştırılmasının önemi.

11.3. Gelecek Çalışmalar (Future Work)

V2 hedefleri: Canlı harita socket entegrasyonları, mekan sahiplerinin kendi yerlerini "claim" etme özellikleri.

V3 hedefleri: Yorum ve Puanlama analizinin AI ile yapılması.

11.4. Kazanılan Yetkinlikler

- MongoDB GeoSpatial API: İki boyutlu küresel harita endeksleme (2dsphere), konum sorgulama, aggregate pipeline'da mesafe hesaplamaları.
- Fastify Framework: Express'e göre daha performanslı bir Node.js altyapısı kurma.
- RBAC ve JWT: Rol tabanlı erişim kontrolü, özel gizli admin paneli kurulumu.
- Modern UI Geliştirme: Framer Motion, Tailwind CSS ve "Glassmorphism" tasarımları.
- Veri Doğruluğu: Seed veri oluştururken API ile gerçeğe uygun koordinat ayıklama bilinci.

KAYNAKÇA

- [1] Node.js Documentation. "Node.js v20". <https://nodejs.org/en/docs/>
- [2] MongoDB Documentation. "Geospatial Queries". <https://www.mongodb.com/docs/manual/geospatial-queries/>
- [3] React Documentation. "React v18". <https://react.dev/>
- [4] Fastify Documentation. "Fastify - Fast and low overhead web framework". <https://www.fastify.io/>
- [5] OWASP Foundation. (2021). OWASP Top 10 — 2021 Edition. <https://owasp.org/Top10/>
- [6] Cagan, M. (2018). Inspired: How to Create Tech Products Customers Love. Wiley.
- [7] Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly.

- [8] Osterwalder, A. & Pigneur, Y. (2010). Business Model Generation. Wiley.
- [9] Web Content Accessibility Guidelines (WCAG) 2.1. (2018). W3C Recommendation. <https://www.w3.org/TR/WCAG21/>
- [10] Leaflet JS. "Leaflet - an open-source JavaScript library for mobile-friendly interactive maps". <https://leafletjs.com/>

EKLER

EK C — Kod Örnekleri

// Örnek 1: Gizli Admin Middleware (RBAC)

```
export const adminOnly = async (request, reply) => {  
  if (request.user.role !== "admin") {  
    return reply.status(403).send({ error: "Sadece yöneticiler bu işlemi yapabilir" });  
  }  
};
```

// Yorum: Sadece token kontrolü yapmak yerine yetki bazlı kontrol sağlayarak /admin/login üzerinden girmeyen tüm atakları reddettik.

// Örnek 2: MongoDB 2dsphere GeoNear Sorgusu

```
const distanceLimit = 5000; // 5 KM
```

```
const pipeline = [  
  {
```

```
    $geoNear: {  
      near: { type: "Point", coordinates: [lng, lat] },  
      distanceField: "distance",  
      maxDistance: distanceLimit,  
      spherical: true,  
      query: { isActive: true, status: "approved" },  
    },  
  ],  
];
```

// Yorum: Milyonlarca kayıt içinden postGIS gerektirmeden "bana en yakın onaylı mekanları getir" sorgusunun tek hamlede çözülmesini sağlayan süper-hızlı çekirdek yapı.

// Örnek 3: Veritabanına Koordinat Kaydederken Düzeltme (Van Gölü Vakası Önlemi)

```
const docs = allPlaces.map((p) => ({  
  name: p.name,  
  location: p.location, // Mutlaka [Boylam, Enlem] formatında olmalı  
  status: "approved",  
  createdBy: user_id  
}));
```

// Yorum: Haritadan [Lat, Lng] gelen veriyi mutlaka GeoJSON standartlarında tersine dizdik. Aksi takdirde mekanlar denizde çıkıyordu.